

Inventra AI

An AI-narrated inventory copilot for Cambodian minimarts — deterministic retail statistics doing the real work, with a language model layered on top to explain them in plain English.

d:/firstwave/inventra

Next.js 16 • React 19

Prisma / SQLite

reviewed 2026-08-18

ROUTES	VIEW SECTIONS	DATA MODELS	SEED CATALOG
1 page · 8 API	8 client-swapped	5 Prisma	15 SKUs, 45d history

01 — EXECUTIVE SUMMARY

What Inventra actually is

A single-page Next.js application built around one seeded shop owner, one product catalog, and a pure-statistics inventory engine — dressed as an "AI-powered" tool because a language model narrates the numbers, not because it computes them.

Inventra presents itself, in its own schema comments, as an *"AI-powered inventory copilot for Cambodian SMEs."* The build log in the repo (worklog.md) confirms the original brief called for Next.js 15, Supabase, and OpenAI; the delivered app substitutes Next.js 16, Prisma/SQLite, and a hosted LLM SDK (z-ai-web-dev-sdk) — a reasonable adaptation to whatever sandbox it was built in, but worth naming because it explains a few of the seams noted in [Findings](#).

Where it's strong

The retail-math core (`lib/inventory.ts`) is genuinely well-built: coherent stockout-probability heuristics, sensible reorder sizing, an honest confidence score, and graceful LLM fallbacks.

Where it's a demo

Every interactive action outside the seed reset — reorder, mark-reviewed, settings save — is toast-only. Nothing user-initiated persists.

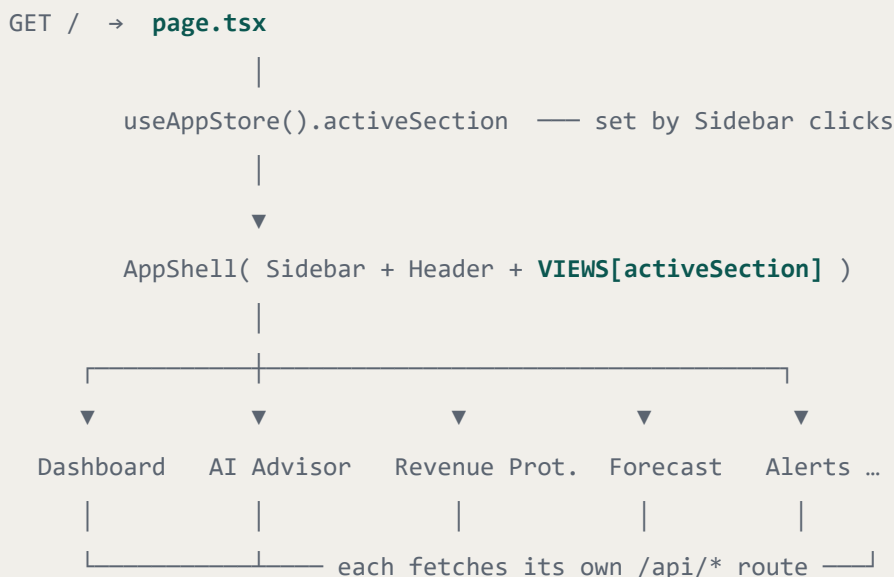
Where it's incomplete

`next-auth` and `next-intl` are full dependencies with zero wiring: no session, no locale routing, no middleware.

02 — ARCHITECTURE

One route, eight views, client-side switching

Despite using the Next.js App Router, Inventra ships exactly one URL-addressable page. "Navigation" is a Zustand store swapping which view component renders inside a persistent shell — closer to a desktop app's tab bar than to routed pages.



Every view is a self-contained client component that calls the shared `useFetch` hook against its own API route on mount, and renders a loading skeleton, an "unseeded" empty state, or the real content. There's no client-side cache layer sitting between views

and the API — each section re-fetches from scratch when mounted, and `@tanstack/react-query` / `@tanstack/react-table`, though installed, aren't used anywhere in the views.

The one structural exception is `AppShell`, which fetches `/api/metrics` independently at the shell level (for the header's revenue-at-risk pill and alert badge) and is also where auto-seeding happens: if the database comes back empty on first load, the shell silently triggers `POST /api/seed` before the user sees anything.

03 — DATA MODEL

Five tables, one implicit tenant

The schema supports multiple users in principle (`Product.userId` foreign key) but every API route resolves the "current user" via `db.user.findFirst()` — there is exactly one shop, always.

MODEL	KEY FIELDS	PURPOSE
User	email, name, businessName	The single seeded shop owner (Sokchea, "Sokchea Mini Mart")
Product	stockQuantity, sellingPrice, costPrice, reorderPoint, unit	Catalog SKU — everything downstream is computed from this + its sales
Sale	productId, quantity, date	One line per sale event; indexed on <code>(productId, date)</code> for the trailing-window queries
Recommendation	revenueAtRisk, suggestedOrderQuantity, confidenceScore, priority, reason	Computed reorder suggestions — regenerated on read, not a durable log
ChatMessage	role, content	Schema exists for persisted advisor chat, but the route never writes to it — history is kept client-side and replayed per request

The math that does the actual work

`src/lib/inventory.ts` is a pure-function library — no I/O, no LLM calls — that turns raw sales rows into everything the UI calls "AI": stockout risk, reorder quantities, confidence scores, alerts, forecasts, and a health score. It's classic retail analytics (moving averages, coefficient of variation, linear trend), not a trained model.

Per-product risk

Every product's daily sales are bucketed into a trailing 30-day series. From that:

```
daysRemaining = stockQuantity / avgDailySales
revenueAtRisk = avgDailySales × (7 - daysRemaining) × sellingPrice [only if stockout falls within 7-day lead time]
```

Stockout probability is a hand-tuned step function over days remaining, boosted by **coefficient of variation** (stdev ÷ mean) so volatile-selling products read riskier at the same stock level than steady ones:

```
≤0 days → 100% <3 days → min(98, 80 + cv×40) <7 days → min(85, 45 + (7-days)×8 + cv×30) <14 days → max(5, 25 - days×1.5) else → max(2, 10 - days×0.4)
```

Reorder sizing & confidence

Suggested order quantity targets a 14-day buffer ($\text{avgDaily} \times 14 - \text{currentStock}$), floored at a minimum 7-day order whenever there's any measurable demand.

Confidence blends two signals: **data volume** (fraction of the 30-day window with an actual sale, 40% weight) and **demand stability** ($100 - \text{cv} \times 60$, 60% weight) — so a product with sparse or erratic sales history is flagged as low-confidence even if the point estimate looks fine.

Forecasting

7/30-day forecasts use the last 14 days of history, estimate a trend slope by comparing the mean of the most recent 3 days against the mean of the oldest 4, then project forward with that trend dampened by a factor of 0.15 so short blips don't dominate the

curve. Running stock is decremented day by day; probability of stockout on any given day rises as projected stock falls below roughly a 3-day demand cushion.

Health score & opportunity feed

Catalog-wide health starts at 100 and is docked per issue — 9 points per critical product, 4 per low-stock, 3 per overstocked, up to 15 for aggregate revenue at risk — landing in one of four bands (Excellent / Healthy / Needs Attention / At Risk). The dashboard's "opportunity feed" mixes three signal types in one ranked list: growth opportunities ($\geq 12\%$ week-over-week), near-term stockout risks, and overstock warnings — capped at eight items so it stays scannable rather than exhaustive.

Notable design choice

Alert generation deliberately suppresses noise: products with more than 12 days of stock and under 8% growth generate no alert at all. The engine is tuned to surface a short, credible list rather than flag everything technically at risk.

05 — AI LAYER

The model narrates; it doesn't calculate

`src/lib/ai.ts` wraps `z-ai-web-dev-sdk` and is used in exactly two places: the free-form advisor chat, and the one-paragraph executive summary on the Insights view. Both send the LLM a compact, pre-computed context block — it never sees raw sales rows and never does arithmetic on the app's behalf.

Grounding context

`buildInventoryContext()` serializes every product's stock, pricing, average daily sales, days remaining, weekly growth, and revenue at risk, plus the top 8 computed recommendations, into a plain-text block injected as context ahead of every prompt — this is what keeps chat answers tied to real numbers instead of hallucinated ones.

Structured output contract

When a user asks the advisor for a reorder list, the system prompt instructs the model to reply with strict JSON (no markdown fences) matching a fixed shape — summary,

overall confidence, and 1–5 line items each with stock remaining, predicted stockout, suggested order, confidence, and revenue protection. `tryParseInsight()` strips code fences and extracts the JSON span defensively; a plain-text reply that doesn't match the shape is passed through untouched rather than treated as an error.

Fallback behavior

If the LLM call throws, both the chat and the insights narrative fall back to rule-based text assembled directly from `generateRecommendations()` — the feature degrades to "the engine's numbers in template sentences" rather than failing outright.

06 — API SURFACE

Eight routes, one response shape

All routes are force-dynamic and follow the same pipeline: load the seeded user's products/sales → run pure functions from `lib/inventory.ts` → return JSON, or `{ seeded: false }` if the catalog is empty.

ROUTE	METHOD	RETURNS
<code>/api/metrics</code>	GET	Dashboard KPIs, opportunity feed, per-product metrics
<code>/api/products</code>	GET	All products with computed metrics + recommendations
<code>/api/alerts</code>	GET	Priority-sorted alert list
<code>/api/forecast</code>	GET	Per-product + aggregate demand curve; <code>?horizon=</code> (default 7)
<code>/api/health</code>	GET	Catalog health score, strengths/risks, category breakdown
<code>/api/insights</code>	GET	Static report + AI-written executive summary (30s max duration)
<code>/api/ai-advisor</code>	POST	Chat reply, Zod-validated body (message 1–2000 chars, ≤20 history turns)
<code>/api/seed</code>	POST	Wipes and rebuilds all demo data deterministically

A ninth route, `/api/route.ts`, returns a bare `{"message": "Hello, world!"}` — a Next.js scaffold leftover with no relation to the app.

07 — VIEWS & SHARED UI

Eight sections behind one sidebar

DASHBOARD

Greeting header, 4 KPI cards, hero "top recommendation" card, weekly revenue chart, opportunity feed, category pie, at-risk snapshot table.

AI ADVISOR

Chat interface with 5 quick-action prompts; renders structured reorder cards when the model returns JSON.

REVENUE PROTECTION

Full sortable at-risk table, filter tabs by priority, top-6 bar chart. "Reorder" button is toast-only.

INVENTORY HEALTH

Animated 0–100 score ring, rule-based strengths/risks text, per-category score grid.

FORECAST CENTER

7/30-day toggle, aggregate demand chart with confidence band, per-product stockout-probability ranking.

BUSINESS INSIGHTS

AI executive summary, revenue drivers, fastest growers, hidden opportunities, dead inventory, cash-flow actions.

ALERTS

Grouped by CRITICAL/HIGH/MEDIUM/LOW with financial impact and confidence per item.

SETTINGS

Store profile, theme toggle (the one setting that's real), AI/notification switches, re-seed button.

Shared components

A consistent visual language runs through every view via `src/components/shared/`: `KpiCard` (stat + trend arrow), `badges.tsx` (`PriorityBadge`, `StatusDot`, `ConfidenceMeter`, `RevenueImpact`), `SectionHeading`, and matched skeleton/empty-state components for every loading and zero-data condition. Charts run entirely on `Recharts`.

08 – SEED DATA

A deterministic Cambodian minimart

`src/lib/mock-data.ts` defines 15 SKUs across six categories — beverages, snacks, staples, condiments, dairy, personal care — priced to match real Cambodian retail (Coca-Cola 1.5L, Anchor Beer, Mama instant noodles, 5kg jasmine rice, M150 energy drink).

Sales history is synthesized with a seeded **Mulberry32 PRNG** (seed derived from each product's id/name length), so re-seeding always reproduces the identical 45-day history — deliberate, for consistent demos. Each product carries a hand-set `{base, trend, noise}` demand profile: bottled water trends up (simulating rising demand for the forecast demo), energy drinks and travel-size shampoo trend down (simulating slow movers), and a 1.35× weekend boost is applied across the board.

09 – STACK & CONFIGURATION

What's actually running



Notable in `next.config.ts`: `typescript.ignoreBuildErrors: true` and `reactStrictMode: false` — both deliberate, but both mean type errors and effect double-invocation issues won't surface at build time. `output: "standalone"` plus the included `Caddyfile` point to a self-hosted deployment rather than Vercel.

`.env` defines a single variable, `DATABASE_URL`, pointing at the local SQLite file.

File layout

```
src/ └─ app/ | └─ page.tsx – the only route; renders AppShell + active view | └─  
api/ – 8 route handlers (see §06) └─ components/ | └─ views/ – 8 section components  
| └─ shared/ – app-shell, sidebar, header, kpi-card, badges... | └─ ui/ – shadcn/ui  
primitives └─ lib/ | └─ inventory.ts – the analytics engine (§04) | └─ ai.ts – LLM  
wrapper (§05) | └─ data.ts – single-tenant DB loader | └─ mock-data.ts – seed  
generator (§08) | └─ store.ts – Zustand nav state | └─ format.ts / nav.ts /  
types.ts / db.ts / utils.ts └─ hooks/ – use-fetch, use-mobile, use-toast
```

10 – FINDINGS

What to know before extending this

Auth and i18n are unwired dependencies, not incomplete features

next-auth and next-intl are in package.json but there's no middleware.ts, no auth route, no session provider, no locale segment, and no messages directory. layout.tsx hardcodes lang="en". Every API route trusts db.user.findFirst() with no session check. Treat any reference to "users" as one hardcoded seed user, not a multi-tenant system.

Most write interactions don't persist

"Reorder" (Revenue Protection), "Mark all reviewed" (Alerts), and the entire Settings form are toast-only — nothing is written back to the database. The only write path in the app is POST /api/seed, which destroys and rebuilds everything. ChatMessage exists in the schema but is never written to; chat history lives only in browser state per session.

"AI" is a thin, well-guarded layer over solid statistics

This is a strength, not a criticism: the LLM is used only for two narrow, low-stakes jobs (chat narration, one summary paragraph), both grounded in pre-computed numbers and both backed by rule-based fallbacks if the AI call fails. Extending the app's actual intelligence means extending inventory.ts, not the AI prompts.

The package name in `package.json` is still the generic starter-template name `nextjs_tailwind_shadcn_ts`, and the repo carries only two commits — consistent with a single AI-assisted build pass (per `worklog.md`) rather than iterative development. For a next phase, the highest-leverage work is wiring real auth (the schema already supports it) and persisting the currently-cosmetic user actions, ahead of adding new analytical features.

Inventra AI — codebase audit • generated from source inspection, no external claims